PROYECTO OSCILOSCOPIO + GENERADOR · STM32

Cómo el STM32 funciona como osciloscopio y mide un Bode hasta 10 MHz

Qué hace el hardware del microcontrolador para capturar dos señales a la vez, y cómo un ADC de 1,6 millones de muestras por segundo puede medir la respuesta de un circuito a frecuencias seis veces más altas que su límite de Nyquist, sin perder exactitud.

Resumen en cinco ideas

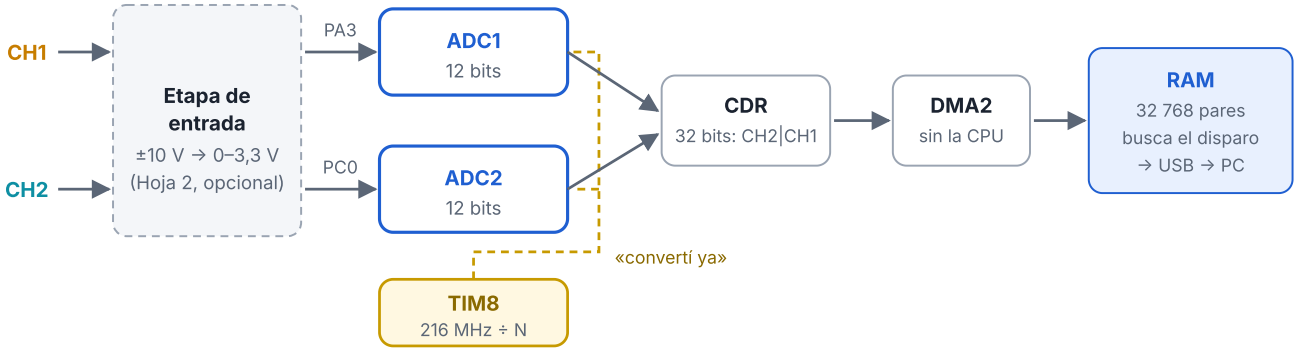
1. **Dos ADC disparados por el mismo reloj** toman CH1 y CH2 en el mismo instante. Esa simultaneidad es la base de todo: del modo XY, de la fase entre canales y del Bode.
2. **El DMA llena la memoria sin la CPU**: el procesador queda libre para el USB, el generador y el AD9833.
3. **El disparo se busca por software** en la memoria, con histéresis. No hace falta hardware de disparo.
4. **El Bode mide un cociente**: H = salida ÷ entrada, con las dos muestras tomadas juntas. Todo lo que afecta igual a los dos canales se cancela.
5. **Arriba de ~635 kHz se submuestrea**: la señal se «pliega» a una frecuencia baja que el ADC sí puede ver, conservando la amplitud y la fase. La frecuencia se estima de los mismos datos.

1. Parte 1 · El STM32 como osciloscopio
2. Parte 2 · Cómo se mide un punto del Bode
3. Parte 3 · Más allá de Nyquist: el submuestreo
4. Parte 4 · Por qué la medición es confiable
5. Parte 5 · ¿Y los 12,5 MHz?
6. Para la exposición

Números para la NUCLEO-F767ZI (núcleo a 216 MHz). Entre paréntesis, la NUCLEO-F429ZI cuando cambia. Versión 0.9.0 del firmware y la app.

Parte 1 · El STM32 como osciloscopio

Un osciloscopio digital necesita cuatro cosas: **muestrear** la tensión a un ritmo exacto, hacerlo en **dos canales a la vez**, **guardar** las muestras sin perder ninguna y **mostrar** la parte que interesa alrededor del disparo. El STM32 resuelve las tres primeras con periféricos de hardware, sin que la CPU intervenga muestra por muestra. La cuarta la hace la app en la PC.



La cadena de captura. Las cajas con borde azul son periféricos del STM32; la etapa de entrada es la placa adicional (sin ella, las señales van directo a los pines y tienen que estar entre 0 y 3,3 V).

1. El reloj de muestreo: un timer

El **TIM8** cuenta pulsos de 216 MHz y cada N pulsos genera un evento (TRGO) que le ordena al ADC convertir. La frecuencia de muestreo es entonces exacta y estable, porque sale del cristal de la placa:

$$f_s = 216\ \text{MHz} \div (\text{PSC} \cdot \text{ARR}) \quad \text{por ejemplo, ARR} = 216 \rightarrow f_s = 1\ \text{MS/s exacto}$$

El firmware calcula PSC y ARR para la f_s pedida y le devuelve a la app la que realmente quedó, que es la que se usa en todas las cuentas.

2. El ADC: 12 bits y 1,59 MS/s

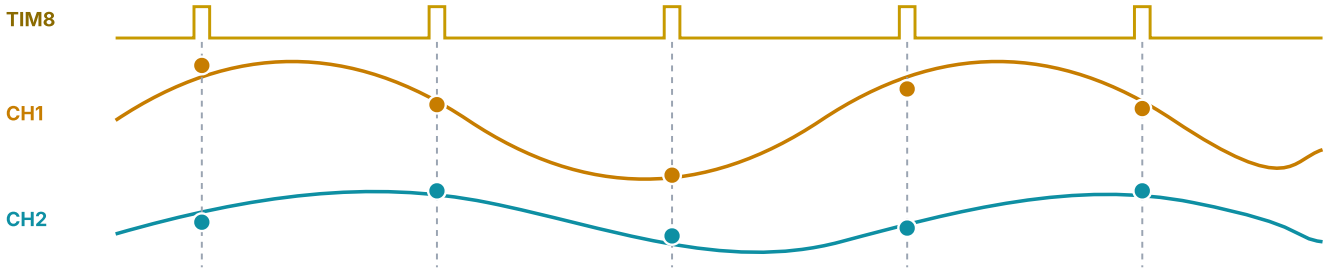
Cada ADC es de **aproximaciones sucesivas** (SAR): primero **muestrea** (conecta el pin a un capacitor interno durante un tiempo) y después **convierte** esa tensión retenida en 12 ciclos de su reloj. El reloj del ADC es 108 MHz ÷ 4 = **27 MHz** (la hoja de datos permite hasta 36 MHz). Con el tiempo de muestreo mínimo de 3 ciclos:

$$t_{\text{conversión}} = (3 + 12)\ \text{ciclos} \div 27\ \text{MHz} = 0{,}56\ \mu\text{s} \rightarrow f_{s\ \text{máx}} = 27\ \text{MHz} \div 17 \approx \mathbf{1{,}59\ MS/s}$$

Se usan 17 ciclos en vez de 15 para dejar margen (en la F429ZI, con 21 MHz, da 1,24 MS/s). Cuando se muestrea más lento, el firmware elige el **tiempo de muestreo más largo que entra** en el período (3 a 480 ciclos): el capacitor interno se carga mejor y la lectura es más exacta, sobre todo con fuentes de alta impedancia. Los 12 bits dan 4096 escalones: en 0–3,3 V, **0,8 mV por código**.

3. Dos canales en el mismo instante: el modo dual

El STM32 tiene tres ADC. El firmware configura ADC1 (CH1, pin PA3) y ADC2 (CH2, pin PC0) en **modo regular simultáneo**: el ADC1 recibe el disparo del timer y el ADC2 arranca en el mismo ciclo de reloj. Los dos resultados se juntan en un único registro de 32 bits (CDR): CH1 en los 16 bits bajos y CH2 en los altos.



Cada flanco del timer hace que los dos ADC tomen su muestra en el mismo instante. El retardo entre canales es cero y el pequeño temblor del reloj (jitter) es el mismo para los dos.

Esto es lo que permite que el **modo XY** y la **fase entre canales** sean exactos, y es el corazón del Bode (partes 2 y 3).

4. Guardar sin la CPU: el DMA

El **DMA2** copia cada palabra de 32 bits del registro CDR a la RAM apenas está lista, y avanza solo a la posición siguiente. Cuando completa la cantidad pedida, genera una interrupción y el firmware marca la captura como completa. Durante todo ese tiempo la CPU no toca las muestras: sigue atendiendo el USB, el generador, el frecuencímetro y el AD9833. El búfer tiene **32 768 pares** (128 KB; en la F429ZI, 16 384), alineado con la caché del Cortex-M7.

5. El disparo, por software

Un osciloscopio tiene que mostrar siempre la señal «quieta», empezando en el mismo punto de la onda. Para eso el firmware captura **tres ventanas** (la pedida y dos más) y después recorre la RAM buscando el cruce:

- **Se arma** cuando la señal baja de *nivel* − *histéresis* (20 códigos ≈ 16 mV). Así el ruido no dispara.
- **Dispara** cuando, ya armado, la señal sube hasta el *nivel* (al revés para flanco de bajada).
- Se envían *n* puntos empezando *pre* muestras antes del cruce, para ver también lo que pasó antes.
- Si no encuentra el cruce, manda la ventana igual y avisa «sin disparo» (modo Auto); en modo Normal la app la descarta.

6. El envío y la app

La placa contesta `DATA n pre fs disparado` seguido de 4·*n* bytes por el USB Full Speed (12 Mbit/s): 8192 puntos son 32 KB. Mientras se envía una captura, si llega otro pedido, se guarda y se arranca apenas termina el envío. En la PC, cada código se pasa a volts con una recta de calibración y, si se usa la etapa de entrada, se deshace su atenuación:

$$V_{pin} = 3{,}3\ \text{V} \cdot \text{código} \div 4095 \quad \cdot \quad \text{con la etapa de } \pm 10\ \text{V: } V_{pin} = 1{,}65\ \text{V} + 0{,}150 \cdot V_{in}$$

Además, el firmware se protege solo: un **watchdog** reinicia la placa si el programa se cuelga, y si una captura tarda bastante más de lo esperado se rearma sola. La app, por su lado, reconecta si se corta el USB.

	NUCLEO-F767ZI	NUCLEO-F429ZI
Núcleo	Cortex-M7, 216 MHz	Cortex-M4, 168 MHz
Reloj del ADC	27 MHz	21 MHz
Muestreo máximo (por canal, los dos a la vez)	1,59 MS/s	1,24 MS/s
Búfer de captura	32 768 pares (128 KB)	16 384 pares (64 KB)
Puntos por canal enviados a la PC	hasta 8192	
Resolución	12 bits (0,8 mV por código en el pin)	

Parte 2 · Cómo se mide un punto del Bode

El diagrama de Bode muestra cómo un circuito modifica una senoidal según su frecuencia: cuánto la amplifica o atenúa (**ganancia, en dB**) y cuánto la adelanta o atrasa (**fase, en grados**). Las dos cosas juntas forman la **transferencia** H(f), un número complejo:

$$H(f) = V_{salida} \div V_{entrada} \quad \cdot \quad \text{ganancia} = 20 \cdot \log_{10}|H| \quad \cdot \quad \text{fase} = \angle H$$



Paso 1: llevar cada canal a un fasor

Para cada frecuencia del barrido se captura un bloque de los dos canales y se ajusta, por **mínimos cuadrados**, una senoidal de esa frecuencia exacta más una continua:

$$v(t) \approx a \cdot \text{sen}(2\pi f t) + b \cdot \cos(2\pi f t) + c \quad \rightarrow \quad X = a + j \cdot b \quad (\text{amplitud } |X| \text{ y fase } \angle X)$$

El ajuste usa **todas** las muestras (miles), así que promedia el ruido mucho mejor que mirar picos o cruces por cero. Es lo que hace la función `fasor_seno()` en `modelo.py`.

Paso 2: dividir

Con los fasores de entrada (X_1) y salida (X_2), **$H = X_2 \div X_1$**. La división es la clave de la exactitud: cualquier cosa que multiplique por igual a los dos canales desaparece del resultado.

$$X_1 = A \cdot e^{j\varphi} \cdot S \qquad X_2 = A \cdot e^{j\varphi} \cdot H \cdot S \quad \Longrightarrow \quad X_2 \div X_1 = H$$

A: amplitud del generador (no hace falta conocerla). φ: en qué parte de la onda empezó la captura (es distinta cada vez). S: lo que le hace a la señal el muestreador del ADC (atenuación y retardo), igual en los dos canales.

Paso 3: promediar si la salida es chica

Cuando el circuito atenúa mucho, la salida es de pocos códigos del ADC y el ruido pesa. La incertidumbre de la fase de un ajuste es aproximadamente *ruido* · *√(2/n)* ÷ *amplitud*, así que la app la estima con la primera captura y decide cuántas más promediar (hasta 16) para llegar a ~1°. Se promedia **H complejo**, no su módulo: así el ruido se cancela en lugar de sumarse como amplitud falsa.

Las dos fuentes de señal

	Generador interno (DAC)	Módulo AD9833 (DDS)
Cómo genera	El DMA recorre una tabla de 4096 puntos a 1 MS/s	Acumulador de fase de 28 bits con reloj de 25 MHz
Frecuencia	Hasta ~100 kHz en el Bode	$f = M \cdot 25\ \text{MHz} \div 2^{28}$, pasos de 0,093 Hz; Bode hasta 10 MHz
Reloj	El mismo que el ADC: la frecuencia se conoce exacta	Cristal propio, corrido unas ppm: la frecuencia se estima de los datos (parte 3)

Parte 3 · Más allá de Nyquist: el submuestreo

Con 1,59 MS/s, el teorema de Nyquist dice que solo se puede reconstruir una señal de hasta 0,79 MHz, y para un ajuste limpio conviene tener al menos 2,5 muestras por período. Por eso **hasta $f_s/2{,}5 \approx 635$ kHz el Bode mide directo**. ¿Cómo se mide entonces un circuito a 10 MHz?

La idea: no hace falta reconstruir la señal, solo su amplitud y su fase

Si se muestrea una senoidal de frecuencia f a un ritmo f_s, cada muestra vale:

$$\begin{aligned} x[k] &= A\cdot\cos(2\pi\, f\, k \div f_s + \varphi) \\ &= A\cdot\cos(2\pi\,(f - m\cdot f_s)\, k \div f_s + 2\pi\, m\, k + \varphi) \\ &= A\cdot\cos(2\pi\, f_a\, k \div f_s + \varphi) \end{aligned}$$

Como $2\pi\cdot m\cdot k$ es un número entero de vueltas, desaparece. Las muestras son **exactamente las mismas** que daría una senoidal lenta de frecuencia $\mathbf{f_a = f - m\cdot f_s}$ (el *alias*), con **la misma amplitud A y la misma fase φ**. El ADC «ve» la señal de 10 MHz como si fuera una de unos cientos de kHz, sin perder la información que el Bode necesita.



Gris: la señal real, que da 2,25 vueltas entre muestra y muestra. Puntos: lo que toma el ADC. Color: la senoidal lenta que pasa por esos mismos puntos. La de CH2 sale más chica y corrida en la misma proporción que la señal real: el cociente CH2/CH1 es el mismo.

Y como **CH1 y CH2 se muestrean en el mismo instante**, los dos se pliegan con el mismo m: el cociente de sus alias es exactamente el H del circuito a la frecuencia real. Solo hay un detalle: si f_a sale negativo, el espectro llega «dado vuelta» y la fase medida tiene el signo cambiado; la app lo corrige tomando el conjugado.

Cómo se elige f_s

La app busca una f_s (de las que el timer puede dar: 216 MHz ÷ N) que deje el alias:

- **cerca de $f_s/4$**: lejos de la continua (que no se puede separar de una señal casi quieta) y de $f_s/2$ (donde el signo de la fase queda ambiguo);
- **lejos de cualquier otra componente plegada**: los armónicos 2f a 5f y la imagen que saca el AD9833 en 25 MHz − f. Si una de ellas cae encima del alias, el ajuste no las puede separar.

Frecuencia real	Método	f_s elegida	Alias que ve el ADC	Captura
500 kHz	directo	1588,2 kS/s (216 MHz ÷ 136)	500 kHz (la misma)	5,2 ms
1 MHz	submuestreo	1341,6 kS/s (÷ 161)	−341,6 kHz ≈ 0,25 f_s	6,1 ms
2 MHz	submuestreo	1588,2 kS/s (÷ 136)	+411,8 kHz ≈ 0,26 f_s	5,2 ms
5 MHz	submuestreo	1542,9 kS/s (÷ 140)	+371,4 kHz ≈ 0,24 f_s	5,3 ms
10 MHz	submuestreo	1479,5 kS/s (÷ 146)	−356,2 kHz ≈ 0,24 f_s	5,5 ms

La frecuencia se mide de los mismos datos

El AD9833 tiene su propio cristal de 25 MHz, que no está sincronizado con el de la placa y puede estar corrido algunas partes por millón. Parece poco, pero el submuestreo no achica el error absoluto: **20 ppm a 10 MHz son 200 Hz**, y en una captura de 5,5 ms eso corre la fase

$$\Delta\varphi = 2\pi \cdot 200\ \text{Hz} \cdot 5{,}5\ \text{ms} \approx 6{,}9\ \text{rad} \quad \text{(más de una vuelta)}$$

Con ese error un ajuste a la frecuencia «nominal» no encuentra la senoidal. Por eso la app **estima la frecuencia del alias en los datos de CH1**: busca el pico de la FFT (con ventana de Hann e interpolación entre bins) y después lo afina buscando la frecuencia que maximiza la amplitud del ajuste (búsqueda de sección áurea). Esa misma frecuencia se usa para CH1 y CH2. Además, «Medir reloj...» en el panel del AD9833 mide el cristal real del módulo con el frecuencímetro y lo corrige en todas las frecuencias.

Parte 4 · Por qué la medición es confiable

Todas las imperfecciones de la cadena entran en una de dos categorías: las que **afectan igual a los dos canales**, que el cociente cancela, y las que **no**, que hay que controlar de otra forma.

Se cancela en el cociente

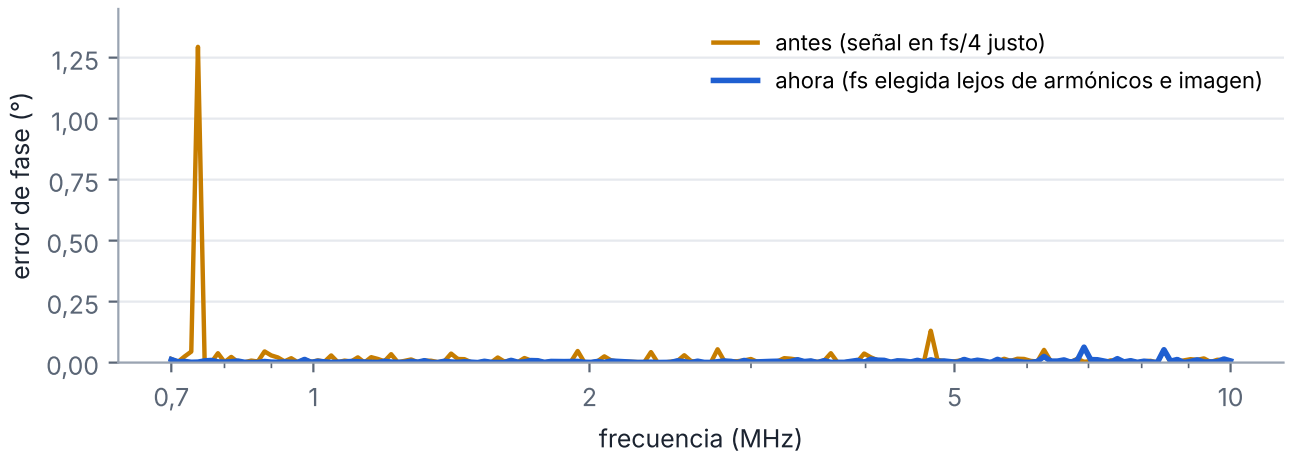
- La amplitud del generador y cómo cae con la frecuencia (el AD9833 pierde 2,4 dB a 10 MHz).
- En qué punto de la onda arrancó cada captura.
- El muestreador del ADC: a algunos MHz el capacitor interno ya no llega a cargarse del todo y atenúa y atrasa, pero ADC1 y ADC2 son idénticos.
- El instante exacto de muestreo y su temblor (jitter): es el mismo flanco del timer para los dos.
- El error del cristal del AD9833: se estima de los datos.

No se cancela (y cómo se controla)

- **Diferencias entre canales** (pines, cables, puntas, tolerancias de la etapa de entrada): cables cortos e iguales en MHz.
- **Acople entre CH1 y CH2**: medimos un piso de unos −55 dB. La app lo detecta y avisa que debajo de ese nivel la medición ya no es del circuito.
- **Otras componentes plegadas** (armónicos, imagen del DDS): se eligen f_s donde caen lejos.
- **Ruido**: ajuste sobre miles de muestras y promedio de varias capturas.

Verificación por simulación

Se simuló el Bode completo de un pasabajos de 100 kHz entre 0,7 y 10 MHz, con una senoidal distorsionada (3.er armónico a −30 dB), la imagen del AD9833 a su nivel real, ruido y la f_s que realmente da el timer. La curva naranja es cómo elegía f_s la versión 0.9.0 (alias justo en $f_s/4$, donde el 3.er armónico cae encima); la azul, la versión de esta revisión.



Error de fase del Bode simulado respecto del valor teórico. Con la elección nueva queda por debajo de 0,1° en todo el rango (y el de ganancia por debajo de 0,05 dB); antes había un pico de 1,3°.

Verificación con circuitos reales (NUCLEO-F767ZI + AD9833)

Circuito	Qué se vio
RC pasabajos (4,7 kΩ + 10 nF)	Pendiente de −20 dB/década y fase hacia −90°, como predice la teoría. En MHz aparece el piso de acople entre canales (unos −55 dB), que la app reconoce y marca.
RL pasaaltos (1 mH + 220 Ω)	La resistencia del alambre de la bobina (escalón de −19 dB en baja frecuencia) y su autorresonancia en ~1,8 MHz.
RLC serie pasabanda	Pico en 100 kHz con Q ≈ 1,6 y la autorresonancia de la bobina como rechazo en 1 MHz. El modelo ajustado queda a 0,3 dB y 1,6° de la medición.

De la curva al modelo

Con los puntos medidos, la app ajusta una transferencia H(s) = N(s) ÷ D(s) de orden 1 a 3 (método de Sanathanan–Koerner: mínimos cuadrados lineales repetidos, con pesos robustos para que un punto malo no arruine el ajuste), elige el modelo más simple que explica la curva y de ahí saca polos, ceros, Q, el tipo de filtro y la frecuencia de corte a −3 dB medida desde la meseta que corresponde. Todo eso va al informe PDF.

Parte 5 · ¿Y los 12,5 MHz?

12,5 MHz es el máximo que puede generar el AD9833, no el máximo del Bode. Un DDS reconstruye la senoidal a partir de muestras tomadas con su reloj de 25 MHz, así que también tiene un límite de Nyquist: 25 ÷ 2 = 12,5 MHz. Cerca de ese límite la salida deja de ser una senoidal limpia:

Frecuencia	Muestras del DDS por período	Imagen en 25 MHz − f	Nivel de la imagen respecto de la señal
1 MHz	25	24 MHz	−27,6 dB
5 MHz	5	20 MHz	−12,0 dB
10 MHz	2,5	15 MHz	−3,5 dB
12,5 MHz	**2**	**12,5 MHz (la misma)**	**0 dB: no se pueden separar**

A 12,5 MHz la señal y su imagen están en la misma frecuencia: lo que sale depende de en qué fase cae cada muestra del DDS y no hay una senoidal definida que medir. Por eso **la app limita el Bode a 10 MHz**: ahí la imagen todavía está 5 MHz más arriba y, eligiendo bien f_s, el ajuste la deja afuera. Entre 10 y 12,5 MHz la imagen se acerca cada vez más a la señal y el módulo (que no trae filtro de reconstrucción) la deja pasar casi entera.

Lo que conviene decir en la exposición

«El generador AD9833 llega a 12,5 MHz, que es la mitad de su reloj. El Bode es confiable hasta **10 MHz**: el STM32 muestrea a 1,6 MS/s, pero como las dos entradas se muestrean en el mismo instante y lo que se mide es el cociente entre ellas, se puede submuestrear. La señal se pliega a unos 370 kHz conservando su amplitud y su fase, y la frecuencia se estima de los mismos datos.»

Para la exposición

Un orden posible (Federico y Darío)

1. **El problema:** un osciloscopio y un analizador de Bode con un microcontrolador de bajo costo.
2. **La captura (parte 1):** timer → dos ADC a la vez → DMA → RAM → disparo por software → USB. Mostrar el diagrama de bloques y el de tiempos.
3. **El Bode (parte 2):** fasores por mínimos cuadrados y el cociente $H = X_2 \div X_1$; qué se cancela.
4. **El truco (parte 3):** el submuestreo con la figura de los puntos y la senoidal lenta; la estimación de la frecuencia.
5. **La validación (parte 4):** simulación y circuitos reales; los límites honestos (acople de −55 dB, 10 MHz y por qué no 12,5).

Preguntas probables

Pregunta	Respuesta corta
¿No viola el teorema de Nyquist?	No: Nyquist habla de reconstruir una señal desconocida. Acá se sabe que es una senoidal de frecuencia conocida (aproximadamente) y solo se mide su amplitud y su fase; el alias conserva las dos.
¿Y el ancho de banda analógico del ADC?	El muestreador atenúa a algunos MHz, pero igual en los dos canales: el cociente lo cancela. Lo que sí hace falta es que la señal no quede enterrada en el ruido.
¿Qué pasa si el circuito tiene ganancia?	Nada cambia en el método; hay que cuidar que la salida no se sature (bajar la amplitud) y que las tensiones entren en la etapa de entrada.
¿Por qué fs/4 y no otra?	Lejos de la continua y de fs/2. Además se corre un poco para que ningún armónico ni la imagen del DDS caiga encima.
¿Qué exactitud tiene?	En simulación, menos de 0,05 dB y 0,25° hasta 10 MHz, aun con la salida 40 dB abajo y ruido. En las mediciones reales, el modelo ajustado queda a ~0,3 dB y ~1,6° de los puntos; el límite práctico es el acople entre canales (~−55 dB).